Submitted Prompts

Submitted Prompt Archive (STAT6207 Assignment 2)

This file collects the **original prompts actually sent to OpenCode** at each stage of this assignment (extracted and verified one by one from `transcripts/`, without rewriting). Each entry notes the corresponding stage, artifacts, and related `bug_log` item.

> Convention: the prompts sent to OpenCode contain only the task itself and do not carry self-checking/correction notes; error checks happen during the session.

Step 0 — EDA (Exploratory Data Analysis)

Artifacts: `report/EDA_report.md`, `results/eda/eda_appliances.png` **Related bug_log**: `#1`, `#2`, `#3` (factual/numerical/argument errors in the EDA report)

```
This is a time-series forecasting task. Do not write a model yet.
Please do an exploratory analysis of @A2_data_release.csv and output a markdown report:
1) the sampling interval, start/end times, rows/columns, and missing values of the data;
2) the statistics of each column (min/mean/max/std);
3) a time-series plot of the target column Appliances (save as eda_appliances.png);
4) determine whether there is an obvious intraday periodicity and give the basis.
Only do analysis and plotting, do not do modeling.
```

Step 1 — Data processing pipeline

Artifacts: `src/data_pipeline.py`, `artifacts/data_windows.npz`, `artifacts/scaler.pkl` **Note**: independent recomputation verified no data leakage / no window out-of-bounds / no look-ahead; no errors found.

```
This is a time-series forecasting project; the target variable is Appliances (one-step-ahead).
Data file: @A2_data_release.csv (10-minute sampling, 1000 rows, 29 columns).

Please write a data_pipeline.py that only does data processing and does not train any model.

[Task]
Turn the series into supervised-learning samples: input = all features of the past L=24 time
steps,
label = the Appliances value of the next time step.

[Requirements]
1. Split in time order: the first 80% is train, the last 20% is test; do not shuffle.
2. Normalization: fit StandardScaler only on the train segment, then use the same scaler to
transform the test segment.
3. Windows must not cross the train/test boundary: train windows come only from the train
segment, test windows only from the test segment.
4. The label of the t-th window comes only from t+1.
5. The date column is used only for sorting and time splitting, not as a numeric feature.
6. Fix the random seed at 42.

[Implementation]
- Put parameters in a constants block at the top: target column name, window length, split
ratio, seed.
- Print: the start/end timestamps of the train/test segments; the raw row counts of each
segment and the sample counts after sliding windows;
  the shapes of X_train/y_train/X_test/y_test; the mean and std of train/test before and after
normalization.
- Save data_windows.npz and scaler.pkl.
- At the end, write the train/test time boundary point in a comment.

Output the script first, then paste the printed results after running it.
```

Step 2 — LSTM initial version (containing the counterexamples `[CODE] #5`, `#7`)

Artifacts: `src/train_lstm.py`, `results/lstm_v1_broken/` **Related bug_log**: `#5` (no validation set/no early stopping → overfitting, RMSE 282), `#7` (using the test set for per-epoch monitoring = test leakage)

```
Based on the data generated in Step 1, write a train_lstm.py that trains an LSTM for one-step
forecasting.

Input files: data_windows.npz (X_train, y_train, X_test, y_test) and scaler.pkl.

[Model]
- PyTorch, nn.LSTM(input_size=number of features, hidden_size=64, num_layers=2,
  batch_first=True, dropout=0.1), with the last time step connected to nn.Linear(64, 1).
- Loss MSELoss, optimizer Adam, lr=1e-3, epochs=50, batch_size=32.
- Fix the random seed with torch.manual_seed(42) to ensure reproducibility.

[Training]
- Train in batches with DataLoader; record train loss and test loss each epoch.
- Save the model as lstm_model.pt after training.

[Evaluation — metrics must be computed on the original scale (Wh)]
- scaler.pkl stores the StandardScaler for the 28 feature columns; take only the mean/scale of
the
  Appliances column and de-normalize the predictions and true y back to the original scale.
- Compute and print MSE, MAE, RMSE on the test set (original scale).
- Additionally compute a naive persistence baseline: directly use the Appliances value of each
window's last time step
  as the next-step prediction, de-normalize it the same way, compute MSE, MAE, RMSE, and print
alongside the LSTM.
- Save the LSTM and naive metrics together as metrics.json.

[Plots]
- lstm_pred.png: test-segment true values vs LSTM predictions (original scale).
- lstm_loss.png: train loss and test loss curves per epoch.
- Add titles, axis units, and legends to both plots.

Output the script first, then paste all printed metrics and generated plots after running it.
```

Step 2b — LSTM fixed version (validation set + early stopping)

Artifacts: `src/train_lstm_v2.py`, `results/lstm_v2/`

```
Improve the LSTM training and model selection. Keep the original train_lstm.py and write a new
train_lstm_v2.py,
with output files uniformly suffixed _v2 (metrics_v2.json / lstm_pred_v2.png /
lstm_loss_v2.png).

[Core changes]
1. Cut a further validation set from the train segment in time order: use the "last 20%" of
train as validation
   and the first 80% for training; cannot shuffle; the test segment must absolutely not
participate in training or model selection.
2. early stopping: train at most 200 epochs, recording train loss and val loss each epoch;
   save the best weights based on the minimum val loss, patience=15, and fall back to the best
weights on timeout.
3. Evaluate finally with the "best epoch" model; compute MSE/MAE/RMSE on the original scale
(Wh),
   and list alongside the naive persistence baseline.

[Compare two inputs under the same early-stopping flow]
A. All 28 features (reuse Step1's X).
B. Univariate: keep only the Appliances history, i.e. X[:, :, 0:1].

[Output]
- metrics_v2.json: record best_epoch, and the MSE/MAE/RMSE of inputs A, B, and naive.
- lstm_pred_v2.png: test-segment true values vs predictions under the best configuration
(original scale).
- lstm_loss_v2.png: train loss and val loss curves per epoch, marking the best epoch.
- Print all key numbers after running.
```

Step 3 — Transformer comparison (Advanced B)

Artifacts: `src/transformer_forecast.py`, `results/transformer/`

```
Write transformer_forecast.py to do the exact same one-step forecasting task as the LSTM with
a Transformer, and compare fairly.

[Data and split — must be exactly the same as train_lstm_v2.py]
- Reuse the split from train_lstm_v2.py: the train segment is the first 800 rows, of which the
last 20% (640-799) is the validation set;
  the test segment is 800-999. The scaler is fit only on sub_train (640 rows); val/test are
only transformed.
- Use the same univariate input as the LSTM's best configuration (Appliances history, i.e.
X[:, :, 0:1]);
  additionally run an all-28-features version for comparison.
- Fix the random seed at 42.

[Model]
- Implement a Transformer encoder with PyTorch (you may use nn.TransformerEncoder, or build
your own,
  with structure referencing the course's MultiHeadSelfAttention / TransformerBlock).
- Positional encoding must be added, because the Transformer itself contains no order
information.
- Structure: Linear projects the input to d_model=64, add positional encoding, pass through 2
TransformerEncoder layers
  (nhead=4, dropout=0.1), take the output of the last time step, and map it to a 1-D
prediction via Linear.
- Loss MSELoss, optimizer Adam, lr=1e-3, up to 200 epochs, early stopping with patience=15
  (the validation-set and early-stopping rules stay consistent with the LSTM).

[Evaluation]
- Use the best-epoch model to compute MSE/MAE/RMSE on the original scale (Wh).
- Read the LSTM and naive results from metrics_v2.json and put them into the comparison table.
- Print the Transformer's parameter count and record the training time (for comparison with
the LSTM).

[Output]
- metrics_transformer.json: containing best_epoch and the metrics of Transformer
(univariate/all features), LSTM, and naive.
- transformer_pred.png: test-segment true values vs Transformer predictions (original scale).
- transformer_loss.png: train/val loss curves, marking the best epoch.
- Print all key numbers after running.
```

Step 4 — Generalizable pipeline (Advanced C)

Artifacts: `src/run_forecast.py`, `results/forecast/metrics_periodB_*.json`

```
[Task: a generalizable time-series forecasting pipeline — adapting to data from "any time
period"]
Write a run_forecast.py that runs directly on any time period of the same dataset, without
needing code changes.

[Command-line arguments]
--csv (data path)
--target (target column, default Appliances)
--window (look-back steps, default 24)
--test-ratio (test-set ratio, default 0.2)
--model (lstm or transformer)
--start / --end (optional, filter data by date)
--tag (output file identifier)

[Automatic data handling (to cope with unseen data)]
- Read the CSV, strip column names; parse date and sort by time;
- If --start/--end are given, trim by time;
- Automatically infer and print the sampling interval;
- Missing values: first ffill, then linear interpolation; print how many missing values were
repaired;
- Drop rows that still contain missing values and report the remaining row count.

[Time-series conventions (must be strictly followed)]
- Split by time, no shuffle; test is the last test-ratio;
- Within train, the last 20% is used as validation;
- The scaler is fit only on the training portion of train; val/test are only transformed;
- Sliding windows do not cross split boundaries; input the past window steps and predict the
next step (t+1).

[Models]
- --model selects LSTM or Transformer (the Transformer must add positional encoding);
- Both use a validation set + early stopping (patience=15).

[Evaluation and output]
- Compute MSE/MAE/RMSE on the original scale and compare against the naive persistence
baseline;
- Output metrics_<tag>.json, pred_<tag>.png, loss_<tag>.png;
- Print the split timestamps, sample counts, and all metrics.
```

Step 4b — Add feature selection to the generalizable pipeline (fixing `[CODE] #6`)

Artifacts: `src/run_forecast.py` (new `--features`), `results/forecast/*_uni.*` **Related bug_log**: `#6`

```
Add a feature-selection capability to run_forecast.py; keep the rest of the logic (time split,
scaler only on train,
sliding windows, early stopping, original-scale metrics, naive comparison) unchanged.

1. Add a command-line argument --features with values {all, univariate}, default all:
   - all: use all numeric columns except date, but exclude the known uninformative columns
rv1, rv2;
   - univariate: use only the target column itself (the target history), i.e. univariate.
2. Print the actually used feature column names and count (for verification).
3. Ensure the target column is always included in the features.
4. Keep the output file naming unchanged, still distinguished by --tag.

After the changes, run the following commands and paste all outputs:
python src/run_forecast.py --csv data/A2_data_release.csv   --model lstm        --features
univariate --tag release_lstm_uni
python src/run_forecast.py --csv data/A2_data_release.csv   --model transformer --features
univariate --tag release_trans_uni
python src/run_forecast.py --csv data/A2_periodB.csv        --model lstm        --features all
--tag periodB_lstm
python src/run_forecast.py --csv data/A2_periodB.csv        --model lstm        --features
univariate --tag periodB_lstm_uni
python src/run_forecast.py --csv data/A2_periodB.csv        --model transformer --features all
--tag periodB_trans
python src/run_forecast.py --csv data/A2_periodB.csv        --model transformer --features
univariate --tag periodB_trans_uni
```

Appendix: transcripts export request

```
Please download and save all records of this session into a folder
```

(Followed by an appended: "in markdown text format")